Python Radio 47: The Hard Way Is The Easy Way

Digital Signal Processing for 9600 baud Packet Radio

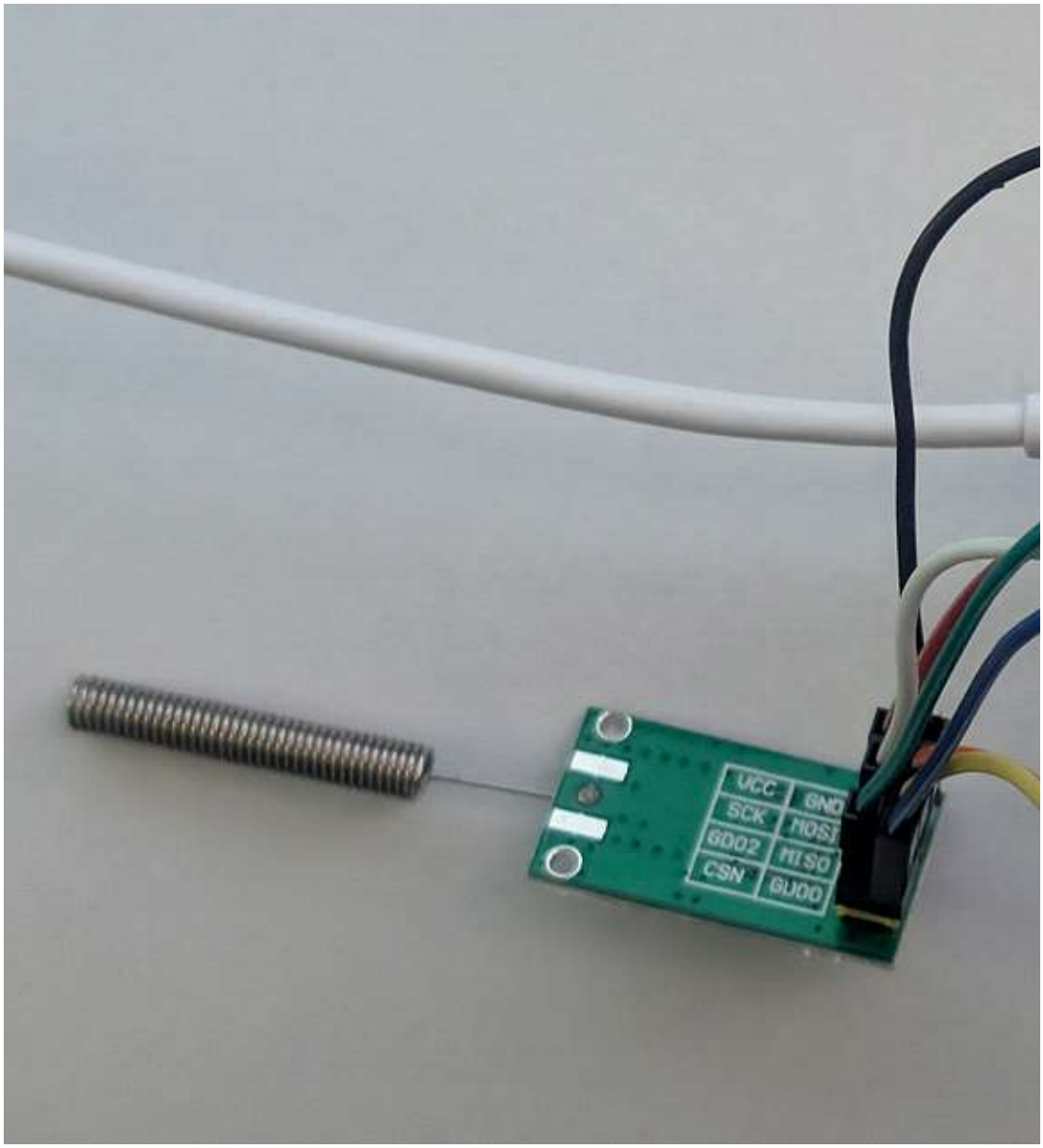
Simon Quellen Field
Follow

13 min read

.

1 day ago

Listen
Share
More



9600 baud packet radio for the 70 cm amateur radio band. Photo by author.

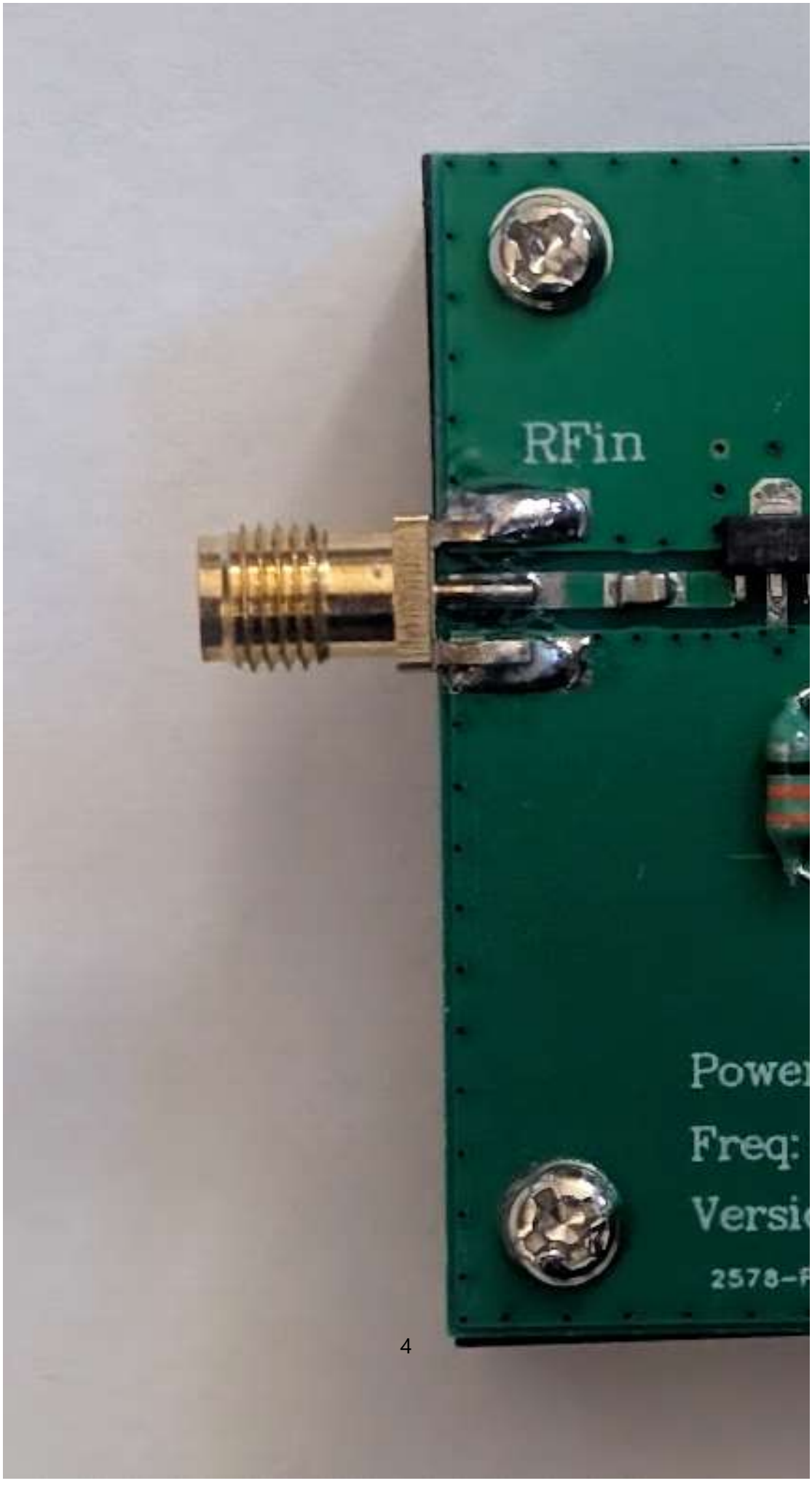
In a previous project, we used the little CC1101 radio module to build a packet radio chat program. It used the built-in packet handling on that radio to send packets at 38.4k baud all the way up to 250k baud (albeit with drastically shorter transmission distances).

It operated in the 70 cm amateur radio band, which includes a license-free portion around 433.92 MHz.

In that band, amateur radio operators use a packet radio mode called AX.25 at 9600 baud. This gives them longer range than faster rates, but is still faster than people can type or read, so it is fine for having a keyboard chat over the radio.

One of the nice things about the 70 cm band is that there are pre-made antennas available without breaking the bank. With these, our little 15 milliwatt radio can talk to pretty much anything it can see (line-of-sight).

And if that isn't good enough, there are cheap little amplifiers you can buy to get 2 watts from a 12-volt power supply or batteries:


RFin
Powe
Freq:
Versi
2578-F

A cheap little amplifier. All photos by the author.

To use such an amplifier, choose a radio with an SMA connector instead of the little spring antenna:


433M
V2.0
8
7
2
1

The CC1101 with an SMA connector.

You will also want an antenna, but the radio usually comes with a small one. You can use a larger antenna, like a Discone or a Yagi-Uda, and get fantastic range.

The CC1101 radio is about $7 on Amazon as of this writing. I get the ESP32C3 Super Mini computers for about $2.25 each, when I order them 10 at a time from AliExpress. On Amazon, they are three for $14.

So this whole project costs about $20 for the two stations ($10 each).

But now we have a problem. The built-in packet processor in the CC1101 can't format AX.25 packets. It uses the wrong CRC code, for one.

So we have to roll the packet processing ourselves. This is why I say the hard way (using the hardware to do the heavy lifting) is the easy way. Luckily, we know Python.

For the CC1101 driver, I've stripped it down to just what we need for 9600 baud packet radio (well, almost — I left in the 38.4k baud code).

```
from machine import Pin, SPI
from time import sleep, sleep_ms, sleep_us, ticks_ms, ticks_diff

# --- CONSTANTS ---
SRES=0x30; SIDLE=0x36; SRX=0x34; STX=0x35; SFRX=0x3A; SFTX=0x3B;
FSTEST=0x59
TXFIFO=0x3F; RXFIFO=0x3F; RXBYTES=0xFB; MARCSTATE=0xF5; PATABLE=
0x3E; PKTLEN=0x06
IOCFG0=0x02; PKTCTRL0=0x08; PKTCTRL1=0x07; FSCTRL1=0x0B; FREQ2=
0x0D; FREQ1=0x0E
FREQ0=0x0F; MDMCFG4=0x10; MDMCFG3=0x11; MDMCFG2=0x12; MDMCFG1=0x22;
MDMCFG0=0xF8; DEVIATN=0x15; MCSM1=0x17
MCSM0=0x18; FOCCFG=0x19; AGCCTRL2=0x1B; AGCCTRL1=0x1C; AGCCTRL0=
0x1D; MDMCFG1=0x13
MAX_PACKET_LEN = 60; FSCTRL0=0x0C; CHANNR=0x0A; FREND0=0x22; FREND1
=0x21
```

```
BSCFG=0x1A; FSCAL3=0x23; FSCAL2=0x24; FSCAL1=0x25; FSCAL0=0x26;
TEST2=0x2C
TEST1=0x2D; TEST0=0x2E; FIFOTHR=0x03

class CC1101:
    def __init__(self, spi, cs_pin_id, gdo0_pin_id, baud=38.4,
freq_offset=0):
        self.spi = spi
        self.cs = Pin(cs_pin_id, Pin.OUT, value=1)
        self.gdo0 = Pin(gdo0_pin_id, Pin.IN, Pin.PULL_DOWN)
        self.rssi_dbm = -99
        self.reset()
        self.freq_offset = freq_offset
        self.configure_gfsk(baud)
        self.set_frequency_mhz(433.92)
        print(f"CC1101 driver initialized with {baud}k GFSK
config.")

    def _write_reg(self, addr, value):
        self.cs.off()
        self.spi.write(bytearray([addr, value]))
        self.cs.on()
        sleep_us(45)

    def _read_reg(self, addr):
        self.cs.off()
        wbuf=bytearray([addr|0x80,0])
        rbuf=bytearray(2)
        self.spi.write_readinto(wbuf,rbuf)
        self.cs.on()
        sleep_us(45)
        return rbuf[1]

    def _read_status_reg(self, addr):
        self.cs.off()
        wbuf=bytearray([addr|0xC0,0])
        rbuf=bytearray(2)
        self.spi.write_readinto(wbuf,rbuf)
        self.cs.on()
        sleep_us(45)
        return rbuf[1]

    def _strobe(self, cmd):
        self.cs.off()
        self.spi.write(bytearray([cmd]))
        self.cs.on()
        sleep_us(45)

    def _write_burst(self, addr, data):
        self.cs.off()
        self.spi.write(bytearray([addr|0x40]))
        self.spi.write(data)
        self.cs.on()
        sleep_us(45)

    def _read_burst(self, addr, length):
        self.cs.off()
        wbuf=bytearray([addr|0xC0]+[0]*length)
        rbuf=bytearray(len(wbuf))
```

```
        self.spi.write_readinto(wbuf,rbuf)
        self.cs.on()
        sleep_us(45)
        return rbuf[1:]

    def reset(self):
        self._strobe(SRES)
        sleep_ms(2)

    def configure_gfsk(self, data_rate_kbps):
        """Applies a complete, self-consistent register set for the
desired baud rate."""
        self._strobe(SIDLE)

        # Common settings for all packet modes
        self.set_tx_power(0xC0)
        self._write_reg(PKTCTRL1, 0x04)      # Append status
        self._write_reg(IOCFG0, 0x06)        # GDO0 for packet
RX/TX signal
        self._write_reg(MCSM1, 0x30)         # After TX/RX -> IDLE
        self._write_reg(MCSM0, 0x18)         # Auto-calibrate from
IDLE
        self._write_reg(PKTCTRL0, 0x00)      # Fixed length, No CRC
        self._write_reg(PKTLEN, MAX_PACKET_LEN)

        # I couldn't get this to work on the CC1101. I'm setting it
up wrong, and I'll look into that later.
        if data_rate_kbps == 1.2:
            self._write_reg(FSCTRL1,  0x06)
            self._write_reg(FSCTRL0,  0x00)
            self._write_reg(MDMCFG4,  0xF5)
            self._write_reg(MDMCFG3,  0x83)
            self._write_reg(MDMCFG2,  0x13)
            self._write_reg(MDMCFG1,  0x22)
            self._write_reg(MDMCFG0,  0xF8)
            self._write_reg(DEVIATN,  0x15)
            self._write_reg(FOCCFG,   0x14)
            self._write_reg(BSCFG,    0x6C)
            self._write_reg(AGCCTRL2, 0x03)
            self._write_reg(AGCCTRL1, 0x40)
            self._write_reg(AGCCTRL0, 0x92)
            self._write_reg(FREND1,   0x56)
            self._write_reg(FREND0,   0x10)
            self._write_reg(FSCAL3,   0xE9)
            self._write_reg(FSCAL2,   0x2A)
            self._write_reg(FSCAL1,   0x00)
            self._write_reg(FSCAL0,   0x1F)
            self._write_reg(FSTEST,   0x59)
            self._write_reg(TEST2,    0x81)
            self._write_reg(TEST1,    0x35)
            self._write_reg(TEST0,    0x09)
            self._write_reg(FIFOTHR,  0x47)

        elif data_rate_kbps == 9.6:
            print("Applying 9.6 kBaud GFSK settings...")
            s =
{'MDMCFG4':0xF8,'MDMCFG3':0x83,'DEVIATN':0x15,'FSCTRL1':0x06,'AGCCT
RL2':0x03,'AGCCTRL1':0x40,'AGCCTRL0':0x91}
            self._write_reg(MDMCFG4,s['MDMCFG4'])
```

```
            self._write_reg(MDMCFG3,s['MDMCFG3'])
            self._write_reg(DEVIATN,s['DEVIATN'])
            self._write_reg(FSCTRL1,s['FSCTRL1'])
            self._write_reg(AGCCTRL2,s['AGCCTRL2'])
            self._write_reg(AGCCTRL1,s['AGCCTRL1'])
            self._write_reg(AGCCTRL0,s['AGCCTRL0'])
            self._write_reg(MDMCFG2, 0x13)
            self._write_reg(MDMCFG1, 0x22)

        else:
            print("Applying proven 38.4 kBaud GFSK settings...")
            s =
{'MDMCFG4':0xA8,'MDMCFG3':0x93,'DEVIATN':0x35,'FSCTRL1':0x06,'AGCCT
RL2':0x03,'AGCCTRL1':0x40,'AGCCTRL0':0x92}
            self._write_reg(MDMCFG4,s['MDMCFG4'])
            self._write_reg(MDMCFG3,s['MDMCFG3'])
            self._write_reg(DEVIATN,s['DEVIATN'])
            self._write_reg(FSCTRL1,s['FSCTRL1'])
            self._write_reg(AGCCTRL2,s['AGCCTRL2'])
            self._write_reg(AGCCTRL1,s['AGCCTRL1'])
            self._write_reg(AGCCTRL0,s['AGCCTRL0'])
            self._write_reg(MDMCFG2, 0x13)
            self._write_reg(MDMCFG1, 0x22)

        self._strobe(SIDLE)


    def set_frequency_mhz(self, freq_mhz):
        freq_hz = int(freq_mhz * 1_000_000) + self.freq_offset
        freq_reg = int((freq_hz / 26000000) * 65536)
        print(f"Setting calibrated frequency to {freq_hz/1e6:.6f}
MHz")

        self._write_reg(FREQ2,(freq_reg>>16)&0xFF)
        self._write_reg(FREQ1,(freq_reg>>8)&0xFF)
        self._write_reg(FREQ0,freq_reg&0xFF)

    def set_tx_power(self, power):
        self._write_reg(PATABLE, power)

    def set_sync_word(self, w1, w0):
        self._write_reg(0x04, w1)
        self._write_reg(0x05, w0)
    def to_rx(self):
        self._strobe(SIDLE)
        self._strobe(SFRX)
        self._strobe(SRX)

    def send_packet(self, data):
        self._strobe(SIDLE)
        self._strobe(SFTX)
        self._write_burst(TXFIFO, data)
        self._strobe(STX)
        # Wait for TX to finish by waiting for GDO0 to go low
        start_time = ticks_ms()
        while self.gdo0.value() == 0:
            if ticks_diff(ticks_ms(), start_time) > 100:
                self.to_rx()
                return # Timeout
```

```
        while self.gdo0.value() == 1:
            if ticks_diff(ticks_ms(), start_time) > 200:
                self.to_rx()
                return # Timeout
        self.to_rx() # Ensure we are back in RX mode

    def receive_packet(self):
        num_bytes = self._read_status_reg(RXBYTES) & 0x7F
        if num_bytes >= MAX_PACKET_LEN:
            payload = self._read_burst(RXFIFO, MAX_PACKET_LEN)
            status = self._read_burst(RXFIFO, 2)
            rssi_val = status[0]
            if rssi_val >= 128:
                self.rssi_dbm = (rssi_val-256)/2.0-74
            else:
                self.rssi_dbm = rssi_val/2.0-74
            self.to_rx() # Flush and re-enter RX
            return payload
        return None
```

We turn off almost all of the packet handling. No CRC, and packets are fixed in length at 60 bytes. The CC1101 will send a preamble of 32 bits that alternate between 1 and 0 for the receiver to lock onto, and two 8-bit sync words so the decoder can tell where bytes start and stop (in case we start listening in the middle of the preamble). But that's it.

The main.py module handles the chat inputs:

```
import sys, select, math
from machine import Pin, SPI, SoftSPI
from time import sleep
from sys import print_exception
from cc1101 import CC1101
from ax25 import Framer # Import the new framer
from whoami import WhoAmI

w = WhoAmI()

SCK_PIN  = w.sck()
MOSI_PIN = w.mosi()
MISO_PIN = w.miso()
CS_PIN   = w.csn()
GDO0_PIN = w.gdo0()
# AX.25 header is 16 bytes. Max packet is 60. Max payload is 40.
MAX_PAYLOAD_PER_PACKET = 39     # Max payload to stay under 60 byte
CC1101 FIFO limit
MAX_PACKET_LEN = 60

def main():
    print("--- AX.25 Chat (Proven GFSK Driver) ---")
    my_call = input(f"Enter YOUR callsign-SSID (default:
{w.callsign()}): ").strip().upper() or w.callsign()
```

```
    dest_call = input(f"Enter DESTINATION callsign-SSID (default:
{w.destcall()}): ").strip().upper() or w.destcall()

    print("-" * 36)
    print(f"Chat ready. My Call: {my_call} | Destination:
{dest_call}")
    print("-" * 36)

    spi = SoftSPI(baudrate=1000000, sck=Pin(SCK_PIN),
mosi=Pin(MOSI_PIN), miso=Pin(MISO_PIN))
    radio = CC1101(spi, CS_PIN, GDO0_PIN, w.baud(),
w.freq_offset())

    framer = Framer(source_call=my_call, dest_call=dest_call)

    poller = select.poll()
    poller.register(sys.stdin, select.POLLIN)

    input_buffer = ""
    is_listening = False
    print(f"You ({my_call})> ", end="")

    while True:
        if not is_listening:
            radio.to_rx()
            is_listening = True

        received_data = radio.receive_packet()
        if received_data:
            is_listening = False
            parsed_frame = framer.parse_ui_frame(received_data)
            # Check if the parsed frame is valid and addressed to
us
            if parsed_frame and parsed_frame['dest'] == my_call:
                source, payload = parsed_frame['source'],
parsed_frame['payload']
                sys.stdout.write('\r' + ' ' * (len(input_buffer) +
15) + '\r')
                print(f"[{source}] {payload}")
                print(f"You ({my_call})> {input_buffer}", end="")

        if poller.poll(1):
            char = sys.stdin.read(1)
            is_listening = False

            if char in ('\n', '\r'):
                if input_buffer:
                    sys.stdout.write('\n')
                    msg_to_send = input_buffer
                    input_buffer = ""
                    total_packets = math.ceil(len(msg_to_send) /
MAX_PAYLOAD_PER_PACKET)
                    for i in range(total_packets):
                        chunk_start = i * MAX_PAYLOAD_PER_PACKET
                        chunk_end = chunk_start +
MAX_PAYLOAD_PER_PACKET
                        msg_chunk =
msg_to_send[chunk_start:chunk_end]
                        frame_to_send =
```

```
framer.build_ui_frame(msg_chunk)
                        padded_frame = frame_to_send + b'\x00' *
(MAX_PACKET_LEN - len(frame_to_send))
                        radio.send_packet(padded_frame)
                        sleep(0.5)
                    print(f"You ({my_call})> ", end="")
            elif char in ('\x7f', '\x08'):
                if len(input_buffer) > 0:
                    input_buffer = input_buffer[:-1]
                    sys.stdout.write('\b \b')
            else:
                input_buffer+=char
                sys.stdout.write(char)

try:
    main()
except KeyboardInterrupt:
    print("\nExiting chat.")
except Exception as e:
    print("An unexpected error occurred:")
    print_exception(e)
```

It collects the characters to send, wraps an AX.25 packet frame around them, and calls the CC1101 driver to send the bits.

Now we come to the heart of the AX.25 protocol, and we build packet frames to send, and decode the ones we receive:

```
# --- CRC-16/X.25 Calculation ---
def crc_x25_update(crc, data):
    crc ^= data
    for _ in range(8):
        if crc & 1:
            crc = (crc >> 1) ^ 0x8408
        else:
            crc >>= 1
    return crc

def calculate_fcs(frame_bytes):
    """Calculates the 16-bit Frame Check Sequence (FCS)."""
    crc = 0xFFFF
    for byte in frame_bytes:
        crc = crc_x25_update(crc, byte)
    fcs = crc ^ 0xFFFF
    return bytearray([(fcs & 0xFF), (fcs >> 8) & 0xFF]) # LSB first

def check_fcs(frame_with_fcs):
    """Verifies the FCS of a received frame. Returns True if
valid."""
    crc = 0xFFFF
    for byte in frame_with_fcs:
        crc = crc_x25_update(crc, byte)
    # A valid frame with correct FCS will result in this "magic
number"
    return crc == 0xF0B8
```

```
# --- Address Encoding/Decoding ---
def encode_address(callsign_ssid, final_address=False):
    parts=callsign_ssid.split('-')
    callsign=parts[0].upper()
    ssid=int(parts[1]) if len(parts)>1 else 0
    if len(callsign)>6:
        raise ValueError("Callsign too long")
    if len(callsign)<6:
        callsign=callsign+' '*(6-len(callsign))
    encoded_call=bytearray([(ord(c)<<1) for c in callsign])
    ssid_byte=(ssid<<1)|0b01100000
    if final_address:
        ssid_byte|=0x01
    return encoded_call+bytearray([ssid_byte])

def decode_address(addr_bytes):
    callsign=""
    for byte_val in addr_bytes[:6]:
        char_code = byte_val >> 1
        if 32 <= char_code <= 126:
            callsign += chr(char_code)
        else:
            callsign += '?'
    callsign=callsign.strip()
    ssid=(addr_bytes[6]>>1)&0x0F
    return f"{callsign}-{ssid}" if ssid>0 else callsign

class Framer:
    def __init__(self, source_call, dest_call):

self.dest_addr=encode_address(dest_call,final_address=False)

self.source_addr=encode_address(source_call,final_address=True)
        self.ax25_flag = 0x7E

    def build_ui_frame(self, payload_str):
        """Builds a complete, compliant AX.25 UI frame with bit-
stuffing and CRC."""
        payload = payload_str.encode('utf-8')
        control_pid = b'\x03\xF0'

        core_frame = self.dest_addr + self.source_addr +
control_pid + payload
        fcs = calculate_fcs(core_frame)
        packet_to_stuff = core_frame + fcs
        print_str = "".join(["".join(hex(x)[2:] + " ") for x in
packet_to_stuff])

        # --- BIT STUFFING --
        stuffed_bits = []
        one_count = 0
        for byte_val in packet_to_stuff:
            for i in range(8):
                bit = (byte_val >> i) & 1
                stuffed_bits.append(bit)
                if bit == 1:
                    one_count += 1
                    if one_count == 5:
```

```
                        stuffed_bits.append(0)
                        one_count = 0
                else:
                    one_count = 0

        # --- NRZI ENCODING ---
        nrzi_encoded_bits = []
        last_level = 1
        for bit in stuffed_bits:
            if bit == 1: # A '1' is "no transition"
                nrzi_encoded_bits.append(last_level)
            else: # A '0' is "transition"
                last_level = 1 - last_level # Flip the level
                nrzi_encoded_bits.append(last_level)

        # --- PACK BITS INTO BYTES ---
        stuffed_bytes = bytearray()
        byte_val = 0
        for i, bit in enumerate(nrzi_encoded_bits):
            if bit:
                byte_val |= (1 << (i % 8))
            if (i + 1) % 8 == 0:
                stuffed_bytes.append(byte_val)
                byte_val = 0

        if len(nrzi_encoded_bits) % 8 != 0:
            stuffed_bytes.append(byte_val)

        pkt = bytearray([self.ax25_flag]) + stuffed_bytes +
bytearray([self.ax25_flag])
        return pkt

    def parse_ui_frame(self, frame):
        """
        Parses a raw buffer, finds the HDLC frame, performs NRZI
decoding,
        de-stuffs it, checks CRC, and decodes the AX.25 content.
        """
        try:
            # 1. Find start and end flags in the raw buffer
            start = -1
            for i in range(len(frame)):
                if frame[i] == self.ax25_flag:
                    start = i
                    break
            if start == -1:
                # print("No start flag")
                return None # No start flag found

            end = -1
            for i in range(start + 1, len(frame)):
                if frame[i] == self.ax25_flag:
                    end = i
                    break
            if end == -1:
                # print("No end flag")
                return None # No end flag found

            stuffed_bytes = frame[start + 1 : end]
```

```
            if not stuffed_bytes:
                # print("No stuffed bytes")
                return None

            sb = ["".join(hex(x)[2:] + " ") for x in stuffed_bytes]
            sb = "".join(sb)

            # --- UNPACK BYTES TO NRZI BITS ---
            nrzi_bits = []
            for byte_val in stuffed_bytes:
                for i in range(8):
                    nrzi_bits.append((byte_val >> i) & 1)

            # --- NRZI DECODING ---
            stuffed_bits = []
            last_level = 1 # Idle state is 1
            for level in nrzi_bits:
                if level == last_level:
                    stuffed_bits.append(1) # No transition means a
'1'
                else:
                    stuffed_bits.append(0) # Transition means a '0'
                last_level = level

            # --- BIT-DE-STUFFING ---
            destuffed_bits = []
            one_count = 0
            for bit in stuffed_bits: # Process the NRZI-decoded
bits
                if one_count == 5:
                    if bit == 0: # This is a stuffed bit, discard
it
                        one_count = 0
                        continue
                    else:
                        # Protocol error: a '1' after five '1's.
                        # This indicates a corrupt frame, but we
can try to reset and continue.
                        one_count = 1
                elif bit == 1:
                    one_count += 1
                else: # bit is 0
                    one_count = 0
                destuffed_bits.append(bit)

            if len(destuffed_bits) < 16 * 8:
                print(f"Short packet: {len(destuffed_bits)} bits")
                return None # Min frame size (addr+ctrl)

            # Truncate to the nearest byte boundary before packing
            if len(destuffed_bits) % 8 != 0:
                destuffed_bits = destuffed_bits[:-
(len(destuffed_bits) % 8)]

            # Pack de-stuffed bits into the final frame for CRC
check
            destuffed_frame = bytearray()
            for i in range(0, len(destuffed_bits), 8):
                byte_val = 0
```

```
                for j in range(8):
                    if destuffed_bits[i+j]:
                        byte_val |= (1 << j) # Reconstruct byte LSB
first
                destuffed_frame.append(byte_val)

            if not check_fcs(destuffed_frame):
                print(f"Bad FCS")
                return None

            core_frame = destuffed_frame[:-2] # Remove the 2-byte
FCS

            if len(core_frame) < 16: # Min core frame size (2 addr
+ 2 ctrl/pid)
                print(f"Short core frame")
                return None

            # --- Address, Control, and PID validation ---
            # The last address field must have the extension bit
set
            if (core_frame[13] & 0x01) != 1:
                print(f"Address extension bit not set correctly")
                return None

            dest=decode_address(core_frame[0:7])
            source=decode_address(core_frame[7:14])

            # Check for UI Frame (Control=0x03, PID=0xF0)
            if core_frame[14] != 0x03 or core_frame[15] != 0xF0:
                print(f"Not a UI frame or has wrong PID")
                return None

            payload = core_frame[16:].decode('utf-8','ignore')
            return {'source':source,'dest':dest,'payload':payload}
        except Exception as e:
            print(f"Exception in parse_ui_frame: {e}")
            return None
```

The AX.25 protocol has some interesting features. Each packet starts and ends with a flag byte of 0x7E. This is binary 0111 1110. Six ones flanked by zeroes.

This lets the decoder know that it is an AX.25 packet, and where it ends.

But this causes a problem. What if there is a 0x7E in our data?

To get around this, the protocol uses something called bit-stuffing. If the encoder ever sees five ones in a row, it adds a 0 bit after that.

The decoder now sees five ones in a row and knows to remove the gratuitous zero bit. This is bit-destuffing.

Another thing the protocol calls for is NRZI encoding of the bits. This helps the receiver tell where bits start and end (called clock recovery). If the data has a long sequence of ones or zeroes, the data looks like a long stretch of just one voltage (either high or low). The "clock" can drift, adding or missing a bit or three.

NRZI stands for Non-Return to Zero Inverted. If the signal goes from zero to one, or from one to zero, that counts as a 1 bit. No transition is a zero bit. This ensures that there are transitions where the receiver can synchronize its clock with that of the transmitter. NRZI is used a lot in things like the USB protocol, and magnetic storage devices like hard drives.

Our AX.25 packet framer does the bit stuffing, the NRZI encoding, and the packing of the bits into bytes for the CC1101 driver. It also adds a checksum (a cyclic redundancy code, or CRC).

The AX.25 protocol can handle either simple one-packet frames (called UI frames) where there is no acknowledgement, or a more involved connection protocol that asks for retransmissions if the packet gets corrupted during transit. I have implemented the simpler part for our chat program, since the person at the other end can ask for a retransmission if the packet gets an error.

We use the whoami.py module like we did in earlier projects, with a few additions. This lets each side configure their radio:

```
class WhoAmI:
    def __init__(self):
        self.me = {}
        try:
            with open("whoami.cfg","rb") as f:
```

```
                line = f.read(1024)
                from json import loads
                self.me = loads(line)
        except OSError as e:
            print("Error reading whoami.cfg:", e )

    def name(self):
        if "name" in self.me:
            return self.me["name"]
        return "Unknown"

    def callsign(self):
        if "callsign" in self.me:
            return self.me["callsign"]
        return "Unknown"

    def destcall(self):
        if "destcall" in self.me:
            return self.me["destcall"]
        return "Unknown"

    def freq_offset(self):
        if "freq_offset" in self.me:
            return self.me["freq_offset"]
        return 0

    def baud(self):
        if "baud" in self.me:
            return self.me["baud"]
        return 38.4

    def fec(self):
        if "fec" in self.me:
            return self.me["fec"]
        return True

    def gdo0(self):
        if "gdo0" in self.me:
            return self.me["gdo0"]
        return None

    def gdo2(self):
        if "gdo2" in self.me:
            return self.me["gdo2"]
        return None

    def csn(self):
        if "csn" in self.me:
            return self.me["csn"]
        return None

    def sck(self):
        if "sck" in self.me:
            return self.me["sck"]
        return None

    def mosi(self):
        if "mosi" in self.me:
            return self.me["mosi"]
```

```
        return None

    def miso(self):
        if "miso" in self.me:
            return self.me["miso"]
        return None

    def ip(self):
        if "ip" in self.me:
            return self.me["ip"]
        return None

    def mask(self):
        if "mask" in self.me:
            return self.me["mask"]
        return None

    def gateway(self):
        if "gateway" in self.me:
            return self.me["gateway"]
        return None

    def dns(self):
        if "dns" in self.me:
            return self.me["dns"]
        return None

    def neo_pin(self):
        if "neo_pin" in self.me:
            return self.me["neo_pin"]
        return None

    def neo_how_many(self):
        if "neo_how_many" in self.me:
            return self.me["neo_how_many"]
        return None

    def set_ip(self, sta):
        if "ip" in self.me:
            sta.ifconfig((self.me["ip"], self.me["mask"],
self.me["gateway"], self.me["dns"]))
```

Our two pals are Alice and Bob, and their whoami.cfg files look like this:

```
{
  "name":"Alice",
  "neo_pin":21,
  "neo_how_many":1,
  "gdo0":4,
  "gdo2":5,
  "csn":3,
  "sck":8,
  "mosi":10,
  "miso":9,
  "baud":9.6,
  "fec":1,
  "freq_offset":-18000,
```

```
  "callsign":"AB6NY",
  "destcall":"KC6PLG"
}
```

for Alice, and

```
{
  "name":"Bob",
  "neo_pin":21,
  "neo_how_many":1,
  "gdo0":10,
  "gdo2":1,
  "csn":15,
  "sck":12,
  "mosi":11,
  "miso":13,
  "baud":9.6,
  "fec":1,
  "freq_offset":0,
  "callsign":"KC6PLG"
  "destcall":"AB6NY"
}
```

The call signs are real (mine is AB6NY) since I was actually transmitting. Change them to your own if you have a license, or to NOCALL if you are using the license-free band at low power. Nicknames that don't look like amateur radio callsigns won't fit in the AX.25 frame.

Alice is configured for the ESP32C3 Super Mini, and Bob for the ESP32S3. Since we are using software SPI bit-banging, you are free to use any general-purpose pins you like.

In use, the chat program looks like this:

```
--- AX.25 Chat (Proven GFSK Driver) ---
Enter YOUR callsign-SSID (default: KC6PLG):
Enter DESTINATION callsign-SSID (default: AB6NY):
-----------------------------------
Chat ready. My Call: KC6PLG | Destination: AB6NY
-----------------------------------
Applying 9.6 kBaud GFSK settings...
Setting calibrated frequency to 433.920013 MHz
CC1101 driver initialized with 9.6k GFSK config.
You (KC6PLG)> Hello?
You (KC6PLG)> Anybody home?
You (KC6PLG)> This is a long message that will get broken up into
shorter packets for transmission.
You (KC6PLG)>
```

Bob is transmitting. Alice sees this:

```
Enter DESTINATION callsign-SSID (default: KC6PLG):
-----------------------------------
Chat ready. My Call: AB6NY | Destination: KC6PLG
-----------------------------------
Applying 9.6 kBaud GFSK settings...
Setting calibrated frequency to 433.902008 MHz
CC1101 driver initialized with 9.6k GFSK config.
[KC6PLG] Hello?
[KC6PLG] Anybody home?
[KC6PLG] This is a long message that will get br
[KC6PLG] oken up into shorter packets for transm
[KC6PLG] ission.
You (AB6NY)>
```

Any number of people can chat at the same time, since the messages tell who sent it and who it's for. A whole class can chat for about $10 a student.